Questão 1. Considere os seguintes algoritmos recursivos que resolvem o mesmo problema em uma entrada de tamanho n:

Algoritmo 1: Divide o problema em 3 partes de tamanho n/4 cada e gasta um tempo adicional O(1) por chamada.

Algoritmo 2: Divide o problema em 3 partes de tamanho n/2 cada e gasta um tempo adicional $O(n^2)$ por chamada.

Algoritmo 3: Divide o problema em 3 partes de tamanho n/3 cada e gasta um tempo adicional de O(n) por chamada.

A complexidade dos algoritmos 1, 2 e 3 é, respectivamente:

A) $\Theta(n^{\log_4 3}), \Theta(n^2), \Theta(n \log n)$

B) $\Theta(n/4), \Theta(n/2), \Theta(n/3)$

C) $\Theta(1), \Theta(n^2), \Theta(n)$

D) $\Theta(n^4), \Theta(n^2), \Theta(n^3)$

E) $\Theta(n^{\log_4 3}), \Theta(n^{\log_2 3}), \Theta(n^{\log_3 3})$

Questão 2. Considere as seguintes funções:

$f(n) = 2^n$

$g(n) = n!$

$h(n) = n^{logn}$

Assinale a alternativa correta a respeito do comportamento assintótico de f(n), g(n) e h(n).

A) $f(n) = O(g(n))$; $g(n) = O(h(n))$.

B) $f(n) = \Omega(g(n))$; $g(n) = O(h(n))$.

C) $g(n) = O(f(n))$; $h(n) = O(f(n))$.

D) $h(n) = O(f(n))$; $g(n) = \Omega(f(n))$.

E) Nenhuma das anteriores.

Questão 3. Dado o trecho de código:

```
int i, j, c;
c = 1;
for (i =1; i < n; i = i*2){
        for (j =1; j <= n; j++){
                c=c+1;
        }
}
```

Assumindo que a instrução c = c+1 é $O(1)$, a expressão que melhor define a ordem de complexidade desse trecho é:

A) O(n log n)

B) $O(\log n)$

C) $O(n)$

D) $O(n^2)$

E) $O(\sqrt{n})$

Questão 4. Para medir o custo de execução de um algoritmo, é comum definir uma função de complexidade f, em que f(n) é a medida de tempo necessário para executar um algoritmo para um problema de tamanho n. Considere as afirmações abaixo sobre funções de complexidade:

I. Se f(n) é uma medida de quantidade de tempo necessário para executar um algoritmo em um problema de tamanho n, então f é chamada função de complexidade de tempo.

II. Se f(n) é uma medida de quantidade de memória necessária para executar um algoritmo de tamanho n, então f é chamada função de complexidade de espaço.

III. A complexidade de tempo não representa o tempo diretamente, mas é estimada pelo número de vezes que determinada operação relevante é executada.

Quais estão corretas?

A) Apenas I.

B) Apenas II.

C) Apenas III.

D) Apenas I e II.

E) I, II e III.

Questão 5. Suponha que, ao invés de dividir em duas partes, foi criada uma versão do mergesort que divida a entrada em quatro partes, ordene cada quarta-parte, e, finalmente, combine essas quatro partes usando um procedimento O(n). A equação de recorrência que descreve o tempo de execução desse algoritmo é:

A) $T(n) = 4*T(n/4) + O(n)$

B) $T(n) = 4*T(n/2) + 2*O(n)$

C) $T(n) = T(n/4) + 4*O(n)$

D) $T(n) = 4*T(n/4) + 4*O(n)$

E) $T(n) = T(n/4) + O(n)$

Questão 6. A complexidade de tempo da questão 5 é:

A) $O(n^2)$

B) $O(n^4)$

C) $O(4*n)$

) $O(n \log n)$

E) $O(n)$

Questão 7. A análise de algoritmos que estabelece um limite superior para o tempo de execução de qualquer entrada é denominada análise:

A) do melhor caso.

B) do caso médio.

) do pior caso.

D) da ordem de crescimento.

E) do tamanho da entrada.

Questão 8. Um algoritmo tem complexidade $O(3m^3 + 2mn^2 + n^2 + 10m + m^2)$. Uma maneira simplificada de representar a complexidade desse algoritmo é:

A) $O(m^3 + mn^2)$.

) $O(m^3)$.

C) $O(m^2)$.

D) $O(mn^2)$.

E) $O(m^3 + n^2)$.

Questão 9. O tempo de execução T(n) de um algoritmo, em que n é o tamanho da entrada, é dado pela equação de recorrência $T(n) = 8T(n/2)+q*n$ se $n > 1$. Dado que $T(1) = p$, e que p e q são constantes arbitrárias, a complexidade do algoritmo é:

A) $O(n)$.

B) $O(n \log n)$.

C) $O(n^2)$.

) $O(n^3)$.

E) $O(n^n)$.

Questão 10. Muitas das recorrências que acontecem na análise de algoritmos de divisão e conquista têm a forma $F(n)=a \cdot F(n/b) + c \cdot n^k$ para F(n) assintoticamente não decrescente, a ,b , $k \in \mathbb{N}$, $a \geq 1, b \geq 2, k \geq 0$ e $c \in \mathbb{R}^+$.

Nessas condições, de acordo com o Teorema Mestre,

- Se $\log a/\log b > k$, então F(n) está em $\Theta(n^{\log a/\log b})$,
- Se $\log a/\log b = k$, então F(n) está em $\Theta(n^k \log n)$,
- Se $\log a/\log b < k$, então F(n) está em $\Theta(n^k)$.

Considere os algoritmos A, B e C, que são descritos, respectivamente, pelas equações de recorrências:

$F_A(n) = 8F(n/4) + n$
$F_B(n) = 4F(n/2) + n^2$
$F_C(n) = 2F(n/4) + n^3$

Dado que $\log_2 2 = 1$, $\log_2 4 = 2$ e $\log_2 8 = 3$, como pode-se comparar a ordem de complexidade Θ dos algoritmos A, B e C?

A) $\Theta(F_A) > \Theta(F_B) > \Theta(F_C)$
B) $\Theta(F_A) < \Theta(F_B) < \Theta(F_C)$
C) $\Theta(F_A) > \Theta(F_B) < \Theta(F_C)$
D) $\Theta(F_A) < \Theta(F_B) > \Theta(F_C)$
E) $\Theta(F_A) = \Theta(F_B) = \Theta(F_C)$

Questão 11. Sejam $T_1(n) = 100n + 15$, $T_2(n) = 10n^2 + 2n$ e $T_3(n) = 0{,}5n^3 + n^2 + 3$ as equações que descrevem a complexidade de tempo dos algoritmos Alg1, Alg2 e Alg3, respectivamente, para entradas de tamanho n. A respeito da ordem de complexidade desses algoritmos, pode-se concluir que:

B) as complexidades assintóticas de Alg1, Alg2 e Alg3 estão, respectivamente, em $O(n)$, $O(n^2)$ e $O(n^2)$.

C) as complexidades assintóticas de Alg1, Alg2 e Alg3 estão, respectivamente, em $O(100)$, $O(10)$ e $O(0,5)$.

D) Alg2 e Alg3 pertencem às mesmas classes de complexidade assintótica.

E) Alg1 e Alg2 pertencem às mesmas classes de complexidade assintótica.

Questão 12. Em relação ao limite assintótico de notação O, atribua V (verdadeiro) ou F (falso) às afirmativas a seguir.

(F) Em uma estrutura de laço duplamente aninhado, tem-se imediatamente um limite superior $O(n^2)$.

(F) Em uma estrutura de laço duplamente aninhado, o custo de cada iteração do laço interno é de limite superior $O(1)$.

(V) Em uma estrutura de laço triplamente aninhado, o custo de cada iteração do laço interno é de limite superior $O(n^3)$.

(V) O limite $O(n^2)$ para o tempo de execução do pior caso de execução aplica-se para qualquer entrada.

(F) f(n) = O(g(n)) é uma afirmação de que algum múltiplo constante de g(n) é de limite assintótico inferior.

Assinale a alternativa que contém, de cima para baixo, a sequência correta.

A) V, V, F, V, F.

B) V, F, V, F, V.

C) F, V, V, F, F.

D) F, F, V, V, F.

E) F, F, F, V, V.

Questão 13 (Bônus). Sejam TA(n) e TB(n) os tempos de execução de pior caso de dois algoritmos A e B propostos para um mesmo problema computacional, em função de um certo parâmetro n.

Dizemos que o algoritmo A é mais eficiente que o algoritmo B assintoticamente no pior caso quando:

A) TA(n) = o(TB(n)).

B) TB(n) = o(TA(n)).

C) TA(n) = O(TB(n)).

D) TB(n) = O(TA(n)).

E) TA(n) = Θ(TB(n)).